

PROJEKT BIOTOP: DIE EVOLUTION EINER SIMULATION

Vom CLI-Legacy-Code zum modularen Retro-SciFi-Labor in C

GEN: 142

POP: 408

FPS: 60

CPU: 3.2 GHz

MEM: 4096 MB

- > Spielregeln
- > main.c - "handgestrickt"
- > Programmstruktur
 - >> Game Loop
 - >> Simulation Loop
- > Spielfeld Topologie
- > GUI - Raylib
- > UX-Design
- > Game Changer Vibe Coding

SIMULATION STATUS: ACTIVE

NETWORK: CONNECTED

MODULES: 12 LOADED

SYSTEM TIME: 14:37:22 UTC

ROLLE:

Senior Software Architekt

TECH STACK:

C, Raylib, Docker, Vibe-Coding

STATUS:

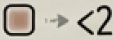
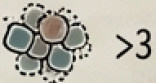
Release 1.0 (Stable)

DAS ZIELBILD: VOM ALGORITHMUS ZUM WETTKAMPF

DIE BASIS

Grundlage: Conway's Game of Life (1970).
Null-Personen-Spiel auf unendlichem Raster.

Regeln:

- Tod durch Einsamkeit (<2) 
- Tod durch Überbevölkerung (>3) 
- Überleben bei 2-3 Nachbarn 

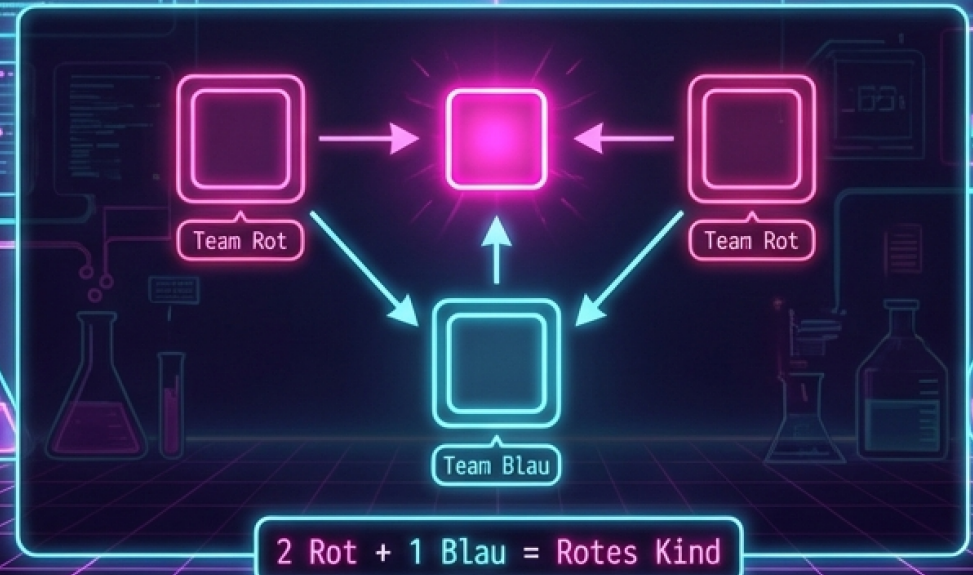
DER TWIST (BIOTOP-REGELN)

Kompetitive Komponente: Team Rot vs. Team Blau.

Geburts-Logik:

- Neue Zelle bei genau 3 Nachbarn.
- Farbe bestimmt durch Mehrheit der Eltern.

Ziel: Strategisches Vorgehen in einer Retro-Labor-Umgebung.



STATUS QUO ANTE: DER LEGACY-MONOLITH

Ausgangslage: Ein einzelnes File (main.c), rein imperativ.

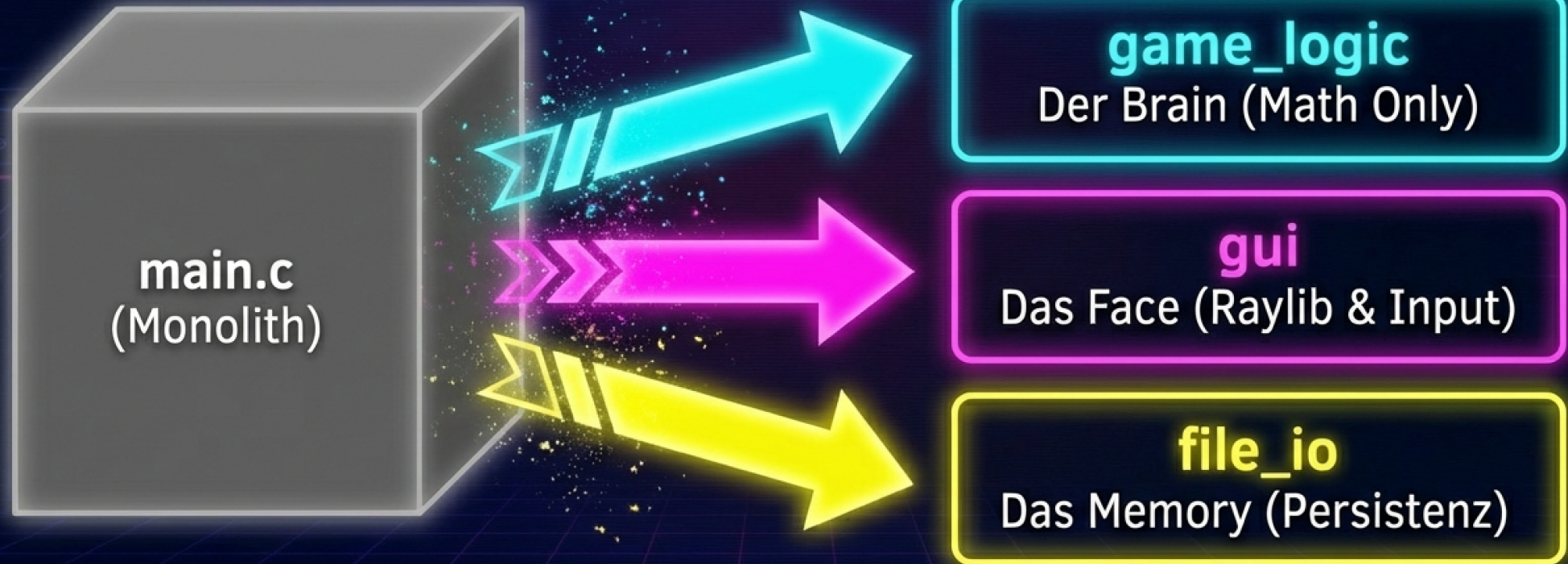
Limitierung: Manuelle Speicherverwaltung, nur CLI-Output (printf).

Code Smell: Hardcodierte Randfall-Prüfung für jede Ecke und Kante.

```
// Sonderfall: i = Eckzelle oben, links
if (i == 0) {
    zellen = current->grid[i + (cols * rows) - 1] + ...
}
else if (i == (cols - 1)) {
    // ... redundante Logik für Ecke rechts oben
}
else if (i == (cols * rows - 1)) {
    // ... noch mehr if-else Kaskaden
}
```

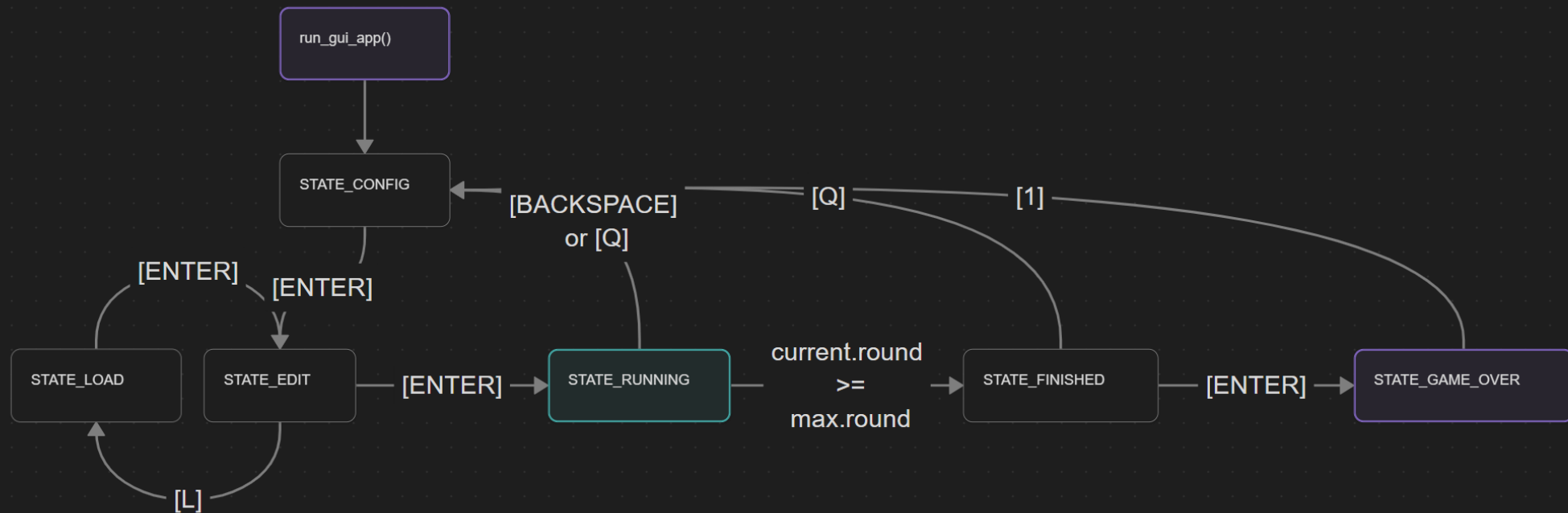
**FUNKTIONAL, ABER
NICHT SKALIERBAR**

PHASE I: ARCHITEKTONISCHE DEKONSTRUKTION



*Separation of Concerns: Die Logik weiß nichts von Pixeln.
Die GUI weiß nichts von Dateisystemen.*

GAME LOOP
gui.c



STATE_RUNNING

Warten(delay_ms)

Welt-Template für
next_gen erstellen

next_gen berechnen
game_logic.h

veraltete gen löschen
free(gui_world)

"Buchhaltung" bzgl.
Rundenzähler und
Population

gui_world wird next_gen

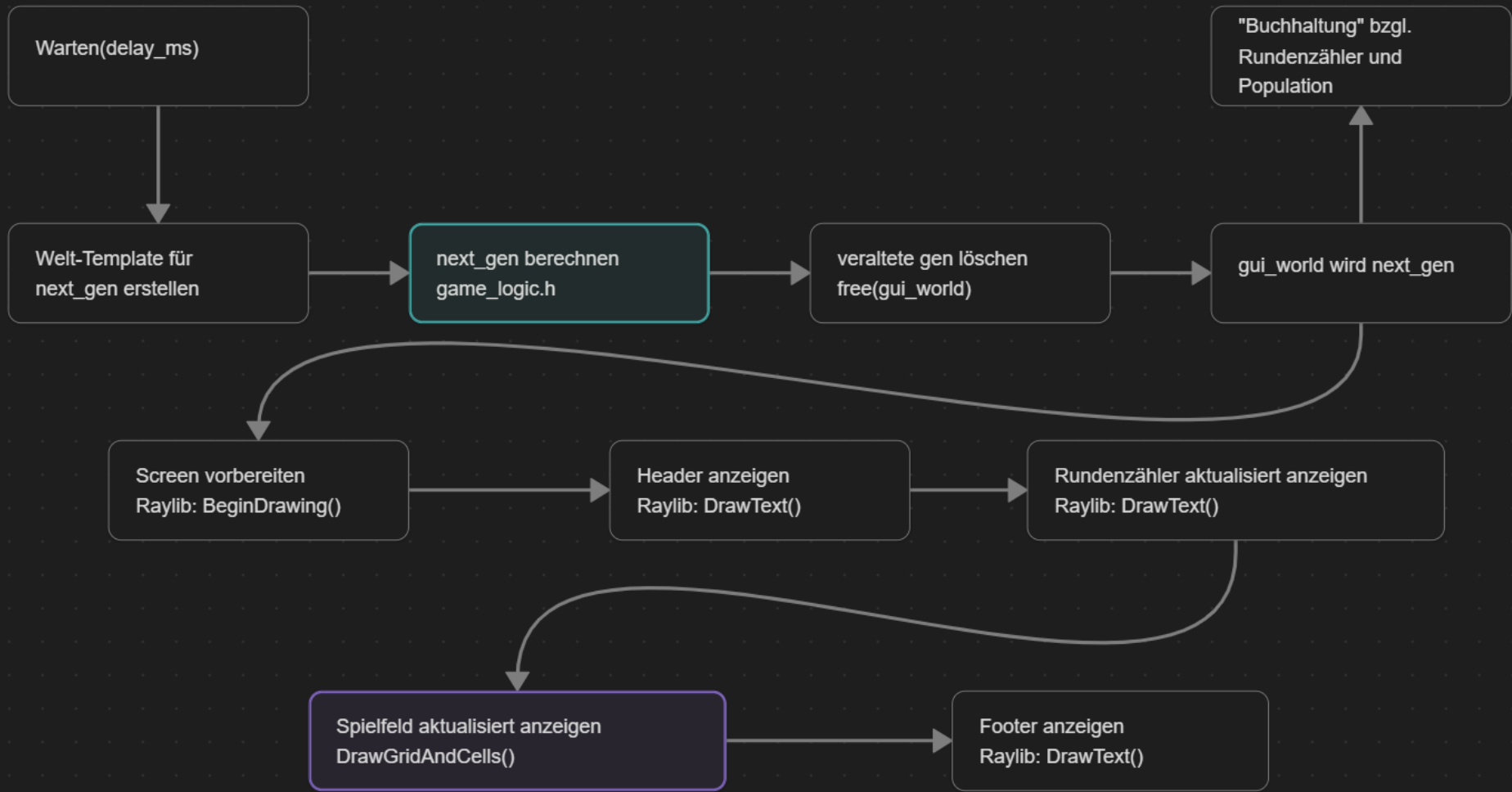
Screen vorbereiten
Raylib: BeginDrawing()

Header anzeigen
Raylib: DrawText()

Rundenzähler aktualisiert anzeigen
Raylib: DrawText()

Spielfeld aktualisiert anzeigen
DrawGridAndCells()

Footer anzeigen
Raylib: DrawText()

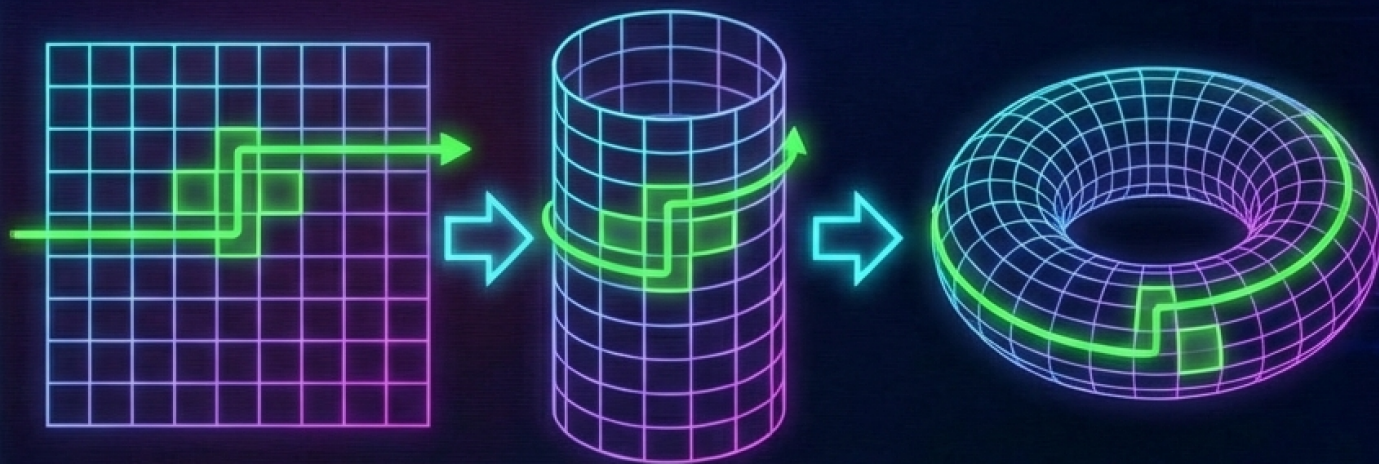


TOPOLOGIE-LÖSUNG: DER TORUS

Problem: Endliches Array, aber unendliche Spiel-Illusion.

Lösung: Modulo-Arithmetik für Weltumrundung.

Formel:
 $(x + \text{width}) \% \text{width}$



Vorher: 150 Zeilen
If-Else für Ränder

Nachher: 1 generische Funktion
`count_neighbors_by_team()`

DIE GUI-REVOLUTION: INTEGRATION VON RAYLIB

- **Entscheidung:** Raylib für OpenGL-Beschleunigung und C-Kompatibilität.
- **Architektur:** Finite State Machine (FSM) steuert den Programmfluss.



UX-DESIGN: DAS DIGITALE BIOLABOR

- **Atmosphäre:** Modern Retro, Dark Mode (#1a1a1a), Neon-Akzente.
- **Wording:** 'Eco-Bloom Cycle' statt 'Round'. Der Nutzer ist Forscher.
- **Rendering:** Padding-Technik statt Linien-Zeichnen für performantes Raster.

CYCLES: 0042

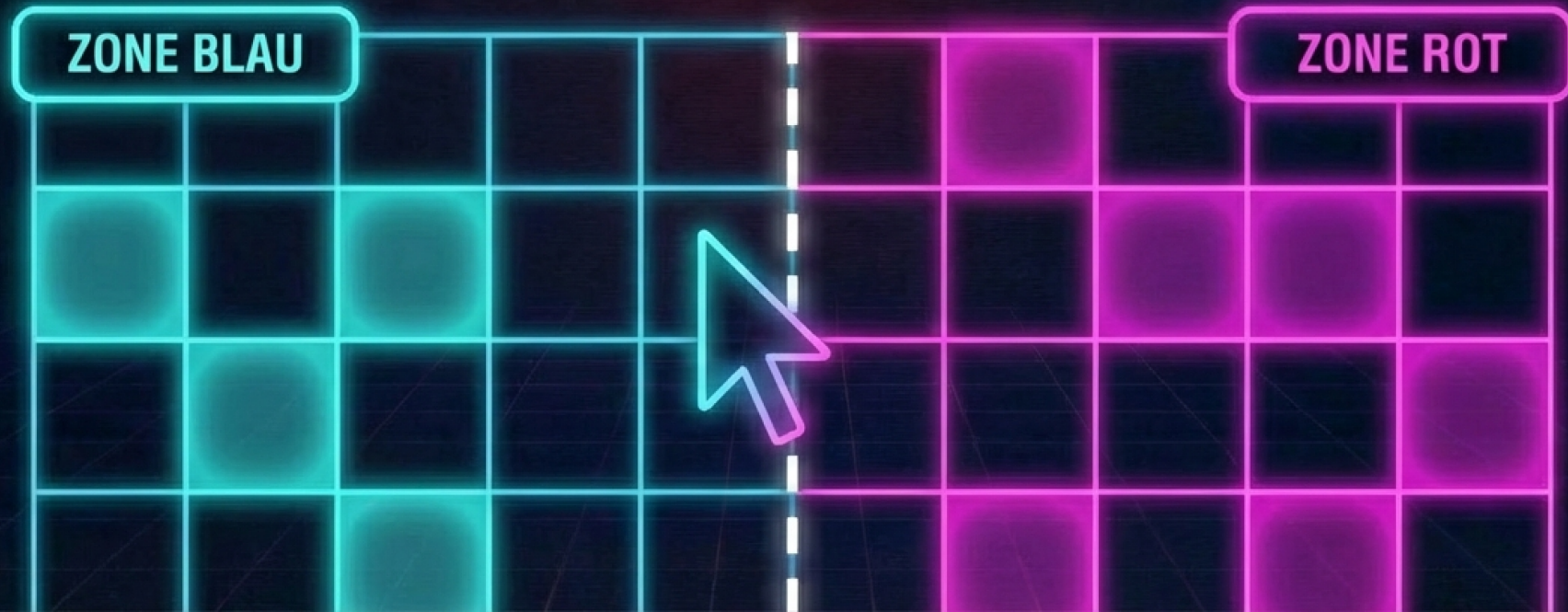
TEAM RED: 150 | TEAM BLUE: 145

[SPACE] PAUSE [S] SAVE



INTERAKTIVER EDITOR & HEMISPHERÄN

- **Fairness durch Geometrie:** Das Spielfeld ist im Editor geteilt.
- **Team Blau:** Linke Hemisphäre. **Team Rot:** Rechte Hemisphäre.
- **Logik:** Coordinate-Mapping in gui.c validiert Input gegen $(\text{cols} / 2)$.



METHODIK: DER VIBE-CODING PROZESS

- **Rollenverteilung:**
- **Mensch** = Senior Architekt & Reviewer.
- **AI (LLM)** = Junior Developer & Implementierer.

